Thin–Walled Structures 204 (2024) 112279

Contents lists available at ScienceDirect

Thin-Walled Structures

journal homepage: www.elsevier.com/locate/tws

Bayesian optimization-based Model-Agnostic Meta-Learning: Application to predict maximum cyclic moment resistance of steel bolted T-stub connections

Yanfei Shen, Mao Li, Yong Li *

Department of Civil & Environmental Engineering, University of Alberta, Edmonton T6H2G7, Canada

ARTICLE INFO

Keywords:
Machine learning
Meta learning
MAML
Bayesian optimization
T-stub connections
Small sample sizes

ABSTRACT

Accurately assessing the maximum moment resistance of steel bolted T-stub connections under cyclic loading is crucial for designing earthquake-resistant structures with such connections. Traditional methods based on design standards like ASCE 41-17 often lack precision. Recently, supervised machine learning techniques, particularly Artificial Neural Network (ANN), have been explored. However, conventional ANNs require substantial data for generalization, which is limited for steel bolted T-stub connections. To address these challenges, this study explores the feasibility of using the Model-Agnostic Meta-Learning (MAML) to predict the maximum cyclic moment resistance of steel bolted T-stub connections. MAML adapts task-specific model parameters rapidly and transfers knowledge across tasks to fine-tune global model parameters, potentially enhancing prediction accuracy with limited data. The MAML model is first optimized using Bayesian optimization with a Gaussian Process model to identify ideal hyperparameters. The optimized MAML model is then compared with two ANN models (one with optimized hyperparameters, another matching MAML's neural network architecture) and ASCE 41-17 method. Results demonstrate the optimized MAML model's superior generalization capabilities, offering a promising approach for steel bolted T-stub connections.

1. Introduction

Steel bolted T-stub connections, as depicted in Fig. 1, offer construction benefits by eliminating the need for on-site welding, thus simplifying both assembly and deconstruction process. Experimental studies have demonstrated that T-stubs in these connections can serve as reliable energy-dissipating elements [1,2]. For seismic-resistant steel moment frame structures incorporating steel bolted T-stub connections, it is crucial to accurately predict the maximum moment resistance (M_{max}) of these connections under cyclic loading. Currently, these predictions largely rely on design standards such as ASCE 41-17 [3]. However, the semi-empirical procedures used in design standards often fall short in prediction accuracy [4], mainly due to the complex behavior of T-stub connections under cyclic loading.

Recently, advanced machine learning techniques have facilitated the development of reliable predicative models for various applications, such as predicting the mechanical properties of engineered cementitious composites [5] (at the material level), the maximum bending moment resistance of high strength steel welded I-section beams [6] (at the member level), and the residual displacement of steel moment-resisting frames using self-centering braces [7] (at the system level). In terms of predicting the structural response of steel beam-column connections, machine learning techniques have also yielded remarkable outcomes. For instance, Jiang and Zhao [8] used six machine learning algorithms to develop regression models for predicting the monotonic strength of stainless steel bolted plate-to-plate connections; Reinosa et al. [9] developed an artificial neural network (ANN) regression model for predicting the initial rotational stiffness of semi-rigid steel top-and-seat connections and top-and-seat double-web connections under monotonic loading; Shah et al. [10] explored various machine learning methods to predict the moment-rotation response of boltless steel connections, aiming to provide efficient and cost-effective alternatives for structural engineering designs. These approaches, rooted in traditional supervised learning, often require a large amount of labeled datasets for training to enhance model generalization—the capability to accurately respond to previously unseen data. However, obtaining large and labeled experimental data is typically impractical, if not impossible. A prominent case is the steel bolted T-stub connections under cyclic loading, for which

* Corresponding author.
E-mail address: yong9@ualberta.ca (Y. Li).

https://doi.org/10.1016/j.tws.2024.112279
Received 29 April 2024; Received in revised form 18 July 2024; Accepted 27 July 2024
Available online 28 July 2024
0263-8231/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

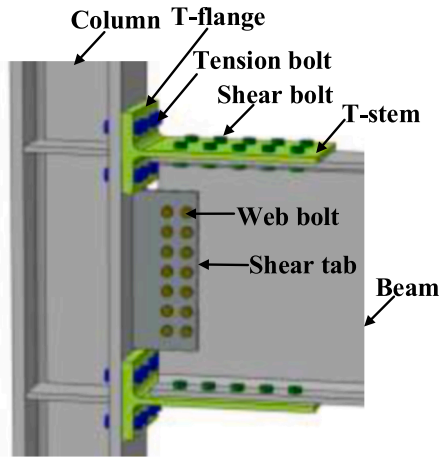


Fig. 1. Steel bolted T-stub connection (excluding column panel zone).

only limited experimental data is available [4].

To better accommodate scenarios with limited data in the field of machine learning, meta-learning has emerged as a focal point of recent research efforts [11–14]. Meta-learning, embodying the "learn to learn" concept, equips machine learning models with the proficiency to assimilate new tasks more effectively. Model-Agnostic Meta-Learning (MAML), initially proposed by Finn et al. [15], has now become a popular meta-learning algorithm due to its favorable performance [16,17]. The principal strategy of MAML involves leveraging small sample sizes to facilitate knowledge transfer across a variety of tasks. It efficiently identifies the model parameters that allow the model to rapidly adapt to new tasks with minimal gradient updates (one or a few steps), using a small amount of labeled data. This leads to the two key advantages of MAML, namely its remarkable generalization capability in scenarios with a small number of samples and its rapid adaptability to new tasks. Despite its potential advantages, the application of MAML for predicting the load-bearing capacity of structural components remains unexplored in the field of structural engineering.

In MAML and other traditional machine learning, hyperparameter tuning emerges as a pivotal step. Hyperparameters shape the model's architecture and dictate how the model learns. These parameters crucially influence the model's capacity to learn and predict. Optimally tuned hyperparameters bolster the model's ability to generalize, averting the pitfalls of overfitting or underfitting, thereby enhancing prediction accuracy. Some researchers [10,15,18–22] rely on empirical knowledge and domain expertise to manually tune various hyperparameters, subsequently selecting the optimal hyperparameter set based on the model's performance. However, this empirical approach is limited as it may identify local optima rather than global optima. To address this issue, grid-search techniques to find the most effective hyperparameters were employed by other researchers [8,23,24]. Grid-search systematically explores a specified subset of hyperparameters by creating a grid of all possible combinations and then evaluates each combination to identify the one that performs best. Although grid-search conducts a thorough exploration of parameter values within predefined limits, its computational demand escalates exponentially with an increase in dimension of the hyperparameter space. This renders grid-search computationally expensive and inefficient for MAML, which often has a high-dimensional hyperparameter space. Recently, Bayesian optimization has become increasingly popular as a solution for scenarios with high-dimensional hyperparameter space [25–27]. Bayesian optimization constructs a probabilistic model of the objective function and utilizes continuously updated prior knowledge to improve the model's accuracy. It selects hyperparameters for evaluation by achieving an optimal balance between the need to explore new areas and the desire to exploit known regions of the search space. Through an iterative refinement process, Bayesian optimization efficiently narrows down the search to identify the best set of hyperparameters.

The study aims to develop a robust predictive model for the maximum cyclic moment resistance of steel bolted T-stub connections, which is a critical indicator of their cyclic behavior. Innovatively, it integrates MAML with Bayesian optimization, leveraging MAML's remarkable generalization capability in scenarios with small sample sizes and enhancing model accuracy through efficient hyperparameter tuning. By addressing the limitations of existing methods, this approach not only improves predictions for steel bolted T-stub connections but also paves the way for predicting the load-bearing capacity of other types of structural components, particularly in cases where experimental data is limited. The remainder of this paper is organized as follows: Section 2 introduces data collection and predictor variable selection conducted prior to training the machine learning models. Section 3 presents the methodology for MAML model development, beginning with gradient-based model parameter updating, followed by the Bayesian optimization approach to identify the optimal group of hyperparameters. Section 4 details the implementation process and specifics of model training and testing. Section 5 displays the performance of the developed MAML model, along with performance comparisons against conventional ANN models and ASCE 41-17 standard method. Finally, interpretability of the machine learning model predictions is conducted before concluding in Section 7.

2. Data collection and feature selection

2.1. Compiled data for bolted T-stub connections subjected to cyclic loading

An experimental dataset on steel beam-to-column bolted T-stub connections subjected to cyclic loading was compiled from the literature [1,2,28–31], with a comprehensive database covering the majority of the collected specimens also presented in [4]. For the tested specimens,

Table 1
Summary of variable statistics.

	Variable	Training samples				Testing samples			
		μ	*SD*	*Min*	*Max*	μ	*SD*	*Min*	*Max*
Predictor	d_{bo} (mm)	21.7	3.0	16.0	35.0	21.5	2.1	20.0	25.4
	f_{ubo} (MPa)	1062.2	51.2	1000.0	1246.0	1067.6	41.4	1000.0	1107.9
	m_o (mm)	55.6	14.8	40.0	93.5	47.9	11.1	43.0	81.0
	n_t	3.6	1.5	2.0	8.0	6.0	2.0	4.0	8.0
	t_{Ts} (mm)	10.7	2.9	6.0	22.0	11.6	2.0	10.0	14.0
	t_{Tf} (mm)	17.0	6.8	10.0	50.0	19.8	4.8	15.0	25.0
	f_{yT} (MPa)	278.0	49.2	235.0	417.1	343.3	47.2	263.2	385.0
	D_b (mm)	367.4	72.8	250.0	600.0	409.5	156.1	250.0	600.0
	W_b (mm)	193.4	16.1	165.1	250.0	184.4	20.3	132.0	200.0
	f_{yb} (MPa)	266.8	44.6	235.0	385.0	372.2	57.1	263.2	423.0
Output	M_{max} (kN·m)	313.8	204.8	150.8	1221.0	523.8	408.9	171.6	1077.5

Table 2
PCC for training sample variables.

	d_{bo}	f_{ubo}	m_0	n_t	t_{Ts}	t_{Tf}	f_{yT}	D_b	W_b	f_{yb}	M_{max}
d_{bo}	1.000										
f_{ubo}	0.365	1.000									
m_0	0.576	0.076	1.000								
n_t	0.539	0.402	-0.243	1.000							
t_{Ts}	0.618	0.676	0.442	0.322	1.000						
t_{Tf}	0.783	0.600	0.540	0.397	0.730	1.000					
f_{yT}	0.343	0.142	0.237	0.424	0.221	0.173	1.000				
D_b	0.706	0.081	0.727	0.214	0.442	0.616	0.339	1.000			
W_b	0.372	0.842	0.445	-0.030	0.688	0.672	-0.057	0.302	1.000		
f_{yb}	0.259	0.032	-0.134	0.572	0.025	0.119	0.770	0.139	-0.246	1.000	
M_{max}	0.686	0.449	0.410	0.447	0.584	0.776	0.149	0.823	0.533	0.137	1.000

the beams and columns connected to the T-stub connections had hot-rolled H-shaped sections (corresponding to "W" sections in AISC [32]). Specimens that did not experience failure due to premature termination of loading protocols or limitations of the testing equipment were excluded from the dataset. This is because the maximum cyclic moments reported do not fully reflect the connections' actual resistance. To further expand the dataset for training purposes, additional data from high-fidelity computational simulations [30,31,33–34] are also included. The resulting dataset, as detailed in Appendix Table A1, comprises 70 samples, with 60 allocated for training and 10 for testing.

2.2. Predictor variable selection

The predictor variables (features) for the machine learning models were initially selected based on the design formulas from ASCE 41-17 [3] for predicting the cyclic maximum strength of T-stub connections. A total of 10 predictor variables were selected, including diameter of the tension bolt shank (d_{bo}), ultimate tensile strength of the bolt (f_{ubo}), the distance from the T-stem face to the centerline of the bolt hole in the first row (m_0), the number of tension bolts on a T-stub (n_t), thickness of the T-stem (t_{Ts}), thickness of the T-flange (t_{Tf}), yield strength of the T-stub (f_{yT}), depth of the beam (D_b), width of the beam (W_b),and yield strength of the beam (f_{yb}). The output variable (label) is the maximum cyclic moment M_{max} for T-stub connections. Statistical characteristics, including mean (μ), standard deviation (*SD*), minimum (*Min*) and maximum (*Max*) values, of the predictor variables and the output variable are summarized in Table 1. The broad range of material properties, geometric dimensions, and the output variable across the samples indicates a diverse dataset. Furthermore, the lack of significant disparities between the training and testing samples ensures that the machine learning model's performance is unbiased, offering reliable predictions across both training and testing datasets.

Subsequently, the Pearson Correlation Coefficient (*PCC*) [35] was employed to measure the correlation among variables. *PCC* quantifies the strength and direction of a linear relationship between two variables on a scale from -1 to +1. In this study, an absolute value of *PCC* above 0.85 is deemed to indicate an excessively high correlation. The *PCC* results for the variables of the training samples are presented in a matrix format in Table 2. It is observed that all predictor variables have *PCC* values below 0.85, indicating no excessively high correlation and thus no need for further elimination of predictor variables. Moreover, it reveals that no features are unrelated or redundant with respect to the output variable. Most predictor variables are moderately or highly correlated with the output variable, underscoring their significance as a predictor variable in the machine learning model to be developed.

3. Methodology for model development

In this study, MAML is fundamentally structured on a neural network. Central to the MAML approach are two phases of learning: the inner loop and the outer loop. In the inner loop, the model is trained on a variety of training tasks, each with its own set of training samples. This phase focuses on adjusting the model's task-specific parameters based on the training samples of each task to optimize its performance on individual task. For each training task, there are also validation samples that are used to evaluate the model's performance after it has been fine-tuned on the training samples. The results from this evaluation guide the gradient calculation for updating the model's across-task parameters (global model parameters) in the outer loop. The outer loop aims to update the model's across-task parameters in a way that enhances its ability to rapidly adapt to new tasks with minimal training.

The following sections detail the methods for gradient and model parameter updates in MAML's inner and outer loops. For comparison, the gradient and model parameter updating methods for conventional ANN are also introduced. The Bayesian optimization technique used to identify the optimal group of hyperparameters for MAML, which is equally applicable to ANN, is then elucidated.

3.1. Gradient and model parameter updates in MAML

The gradient is the partial derivative of the loss function with respect to the model parameters (denoted as $\boldsymbol{\theta}$). It is a vector that points in the direction where the loss function increases most rapidly. Model parameters are optimized during the learning process to accurately map inputs to outputs. The model parameter ($\boldsymbol{\theta}$) consists of vectors of weights and biases for all layers. Specifically, in a fully connected neural network layer, the number of weights in the current layer depends on the number of neurons (C_{ne}) of the layer and the number of neurons (P_{ne}) in the previous layer (providing inputs to the current layer). Each neuron is connected to all the neurons in the previous layer, resulting in a total of $C_{ne} \times P_{ne}$ weights for the current layer. Additionally, each neuron in a layer has its own independent bias.

A key feature of MAML is its implementation of two-level updates: it operates on both inner loop and outer loop updating within each meta epoch (a meta epoch refers to a complete cycle of MAML training).

(1) Inner loop

In the inner loop steps, also known as adaptation steps, MAML performs updates for each training task using its respective training dataset. This process involves calculating the first-order gradient and then optimizing the task-specific model parameters in the direction that reduces loss. Consequently, this results in different model parameters for different training tasks, tailored for rapid adaptation to specific tasks. The formula for updating task-specific model parameters through gradient descent in the inner loop is shown in Eq. (1):

$$\boldsymbol{\theta}^{j}_{task_i} = \boldsymbol{\theta}^{j-1} - lr_{inner} \nabla_{\boldsymbol{\theta}} L^{training}_{task_i}\left(\boldsymbol{\theta}^{j-1}\right) \tag{1}$$

where the superscript j corresponds to the current meta epoch value (1, 2, 3, ...), and subscript i denotes the serial number of the training task. Here, $\boldsymbol{\theta}^{j-1}$ is the across-task (global) model parameters updated in the outer loop (introduced in the following section). lr_{inner} represents the

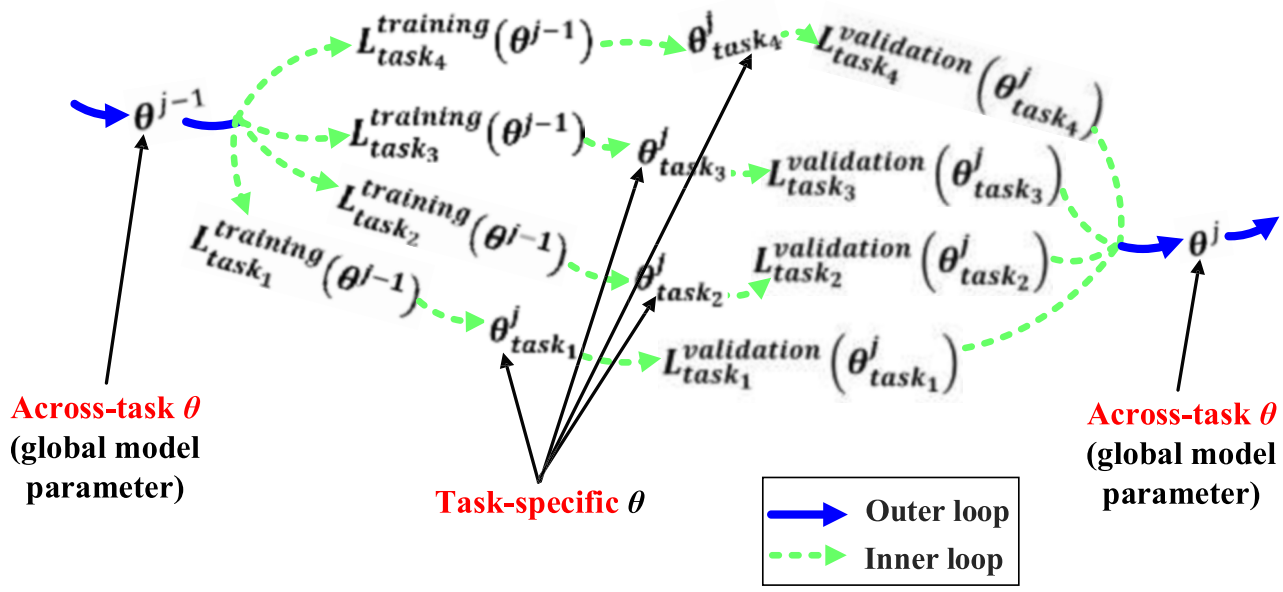


Fig. 2. Inner and outer loop updates in MAML for an illustrative example.

Fig. 3. Illustration of update process in ANN.

learning rate for the inner loop, controlling the step size of model parameter updates during each iteration. $\nabla_{\boldsymbol{\theta}}$ represents the gradient vector, and $L_{task_i}^{training}\left(\boldsymbol{\theta}^{j-1}\right)$ is the training loss function of the i-th task.

(2) Outer loop

In the outer loop, MAML utilizes the model parameters updated in the inner loop along with the validation loss to refine the across-task (global) model parameters ($\boldsymbol{\theta}^{j}$). Through this process, MAML globally updates previous model parameters by evaluating performance across multiple tasks, aiming to find a favorable starting point that enables quick and effective adjustments when encountering new tasks. The across-task model parameters are updated using gradient descent, following Eq. (2):

$$\boldsymbol{\theta}^{j} = \boldsymbol{\theta}^{j-1} - lr_{outer}\nabla_{\boldsymbol{\theta}}\left\{\sum_{meta\ \ batch\ \ size} L_{task_i}^{validation}\left(\boldsymbol{\theta}_{task_i}^{j}\right)\right\} \tag{2}$$

where lr_{outer} is the learning rate for the outer loop, denoting the step size of model parameter updates in each iteration. $L_{task_i}^{validation}\left(\boldsymbol{\theta}_{task_i}^{j}\right)$ is the validation loss function of the i-th task after adaptation in the inner loop.

The updated global model parameter in the outer loop are then passed to the inner loop of the next meta epoch. Fig. 2 illustrates the process of inner and outer loop updates for an example with 4 training tasks and 1 gradient update in the inner loop.

3.2. Gradient and model parameter updates in ANN

In ANN, the training dataset is typically divided into multiple batches. One complete pass of the entire training dataset through the neural network is termed an epoch. In each epoch, after processing each batch, gradient computation and model parameter update are performed (see Fig. 3). The formula for updating model parameters through gradient descent in ANN is as follows:

$$\boldsymbol{\theta}^{j} = \boldsymbol{\theta}^{j-1} - lr\nabla_{\theta}L^{training}\left(\boldsymbol{\theta}^{j-1}\right) \tag{3}$$

where the superscript j (1, 2, 3, ...) represents the batch being processed, lr is the learning rate of ANN, which controls the step size of the parameter updates in each iteration, and $L^{training}\left(\boldsymbol{\theta}^{j-1}\right)$ is the training loss function. It should be noted that the validation loss of ANN is not utilized for updating model parameters during the machine learning process. In contrast, ANN's validation loss is typically employed for hyperparameter tuning.

3.3. Bayesian optimization for identifying optimal hyperparameters

After determining the strategy for gradient and model parameter updates in model training, Bayesian optimization is utilized to identify the optimal hyperparameter set, including neurons per layer, number of layers, inner/outer learning rate, and others. The principle behind Bayesian optimization for fine-tuning hyperparameters involves the following key components:

- Surrogate Model: Bayesian optimization uses a surrogate probabilistic model to approximate the target function. This model can be a Gaussian Process (GP) [36], which predicts the performance of different hyperparameter settings based on past evaluations. The surrogate model provides not only predictions but also uncertainty estimates of these predictions, which are crucial for guiding the optimal hyperparameter search.
- Prior Knowledge: Bayesian optimization incorporates prior beliefs about the objective function's behavior. These beliefs are updated as new data become available. The prior is combined with new

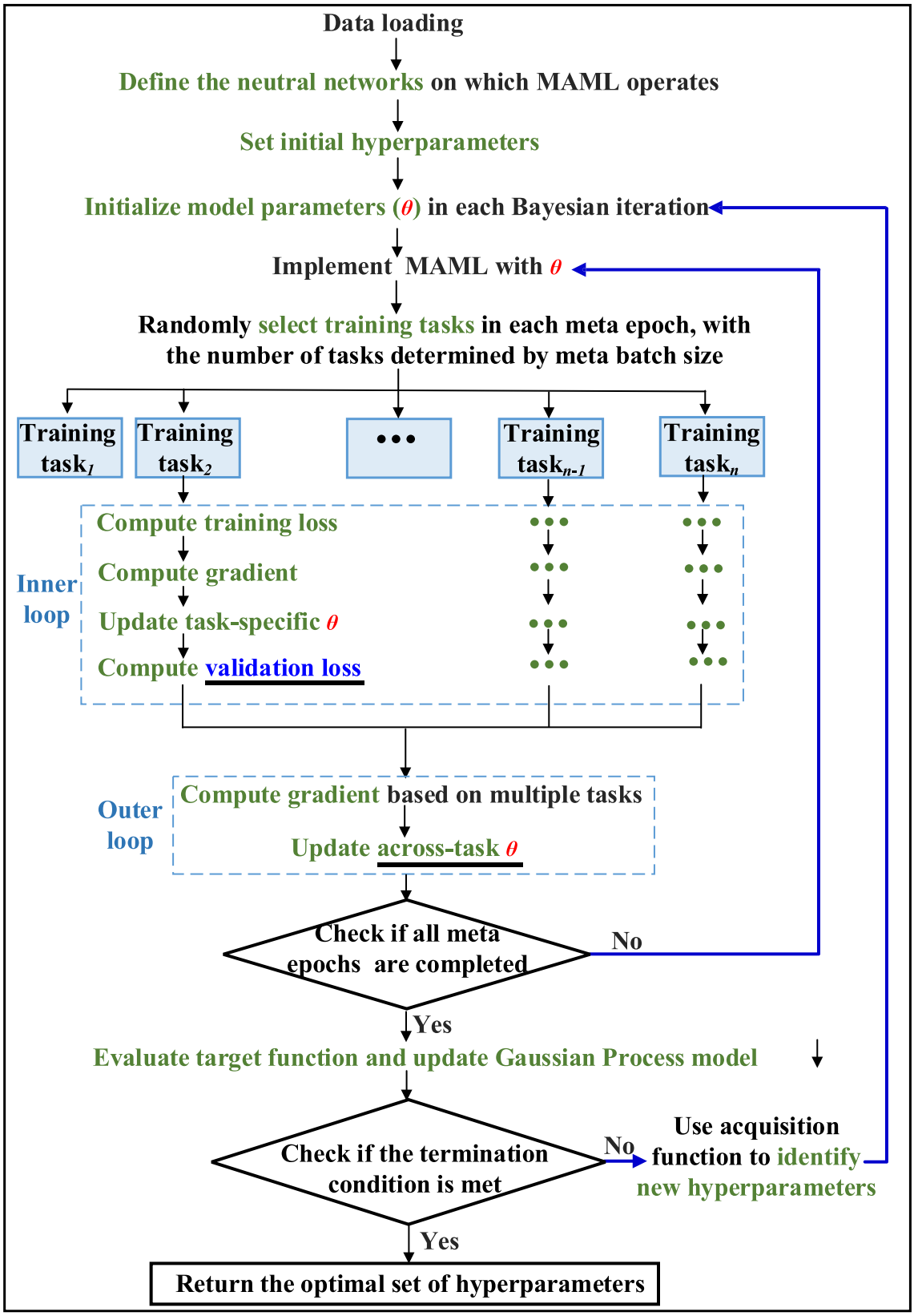


Fig. 4. Illustration of a simplified workflow for Bayesian optimization-based MAML on training tasks.

evidence to form the posterior distribution, which then serves as the new prior for future updates.

- Acquisition Function: Bayesian optimization uses an acquisition function to decide which hyperparameters to evaluate next. This function is designed to find a balance between exploitation (choosing hyperparameters close to the ones that performed well in the past) and exploration (choosing hyperparameters in regions of the search space that have not been thoroughly explored yet). The acquisition

function relies on the surrogate model's predictions and uncertainty estimates to make this decision.

- Iterative Process: The optimization process is iterative. Bayesian optimization suggests a new set of hyperparameters to evaluate by optimizing the acquisition function. After evaluation, the outcome is fed back into the surrogate model, updating it with the new information. This iterative process continues until a stopping criterion is met, such as a maximum number of evaluations or convergence to a satisfactory solution.

In this study, GP is employed as the surrogate model, and the minimum of the average validation loss of training tasks is taken as the target function L_v. Define $\boldsymbol{r}$ as the vector of hyperparameters, with a dimension s (representing the number of hyperparameters). The vector $\boldsymbol{r} = (\varphi_1, \varphi_2, \cdots, \varphi_s)$, where φ_i (i=1,2, ⋯, s) represents each hyperparameter. Given a set of input points $\{\boldsymbol{r}_1, \boldsymbol{r}_2, \cdots, \boldsymbol{r}_m\}$, where m is the number of input points, the target function values at these points are $\boldsymbol{L_v} = [L_v(\boldsymbol{r}_1), L_v(\boldsymbol{r}_2), \cdots, L_v(\boldsymbol{r}_m)]$. In GP, the distribution of $\boldsymbol{L_v}$ follows a multivariate Gaussian distribution, denoted as $\boldsymbol{L_v} \sim \boldsymbol{N}(\boldsymbol{m}, \boldsymbol{K})$, where $\boldsymbol{m}$ is the mean vector and $\boldsymbol{K}$ is the covariance matrix. This study assumes a prior mean vector as a zero vector, and the prior covariance matrix is:

$$K = \begin{pmatrix} k(r_1, r_1) & k(r_1, r_2) & \cdots & k(r_1, r_m) \\ k(r_2, r_1) & k(r_2, r_2) & \cdots & k(r_2, r_m) \\ \vdots & \vdots & \ddots & \vdots \\ k(r_m, r_1) & k(r_m, r_2) & \cdots & k(r_m, r_m) \end{pmatrix} \tag{4}$$

Each element of this covariance matrix represents the covariance between two input points, calculated using a kernel function. This study employs the Squared Exponential Kernel (*SEK*) [39], with the expression for $k(\boldsymbol{r_i}, \boldsymbol{r_j})$ given as:

$$k(\boldsymbol{r}_i, \boldsymbol{r}_j) = exp\left(-\frac{\| \boldsymbol{r}_i - \boldsymbol{r}_j \|^2}{2l^2} \right) \tag{5}$$

where i, j=1,2, ⋯, m; $\|\boldsymbol{r_i} - \boldsymbol{r_j}\|$ is the Euclidean distance between $\boldsymbol{r_i}$ and $\boldsymbol{r_j}$, and l is the length scale parameter controlling the function's smoothness.

The Expected Improvement (*EI*) function is employed as the acquisition function, given by

$$EI(\boldsymbol{r}) = \begin{cases} \{\mu[L_v(\boldsymbol{r})] - L_v^+(\boldsymbol{r}) - \xi\}\phi(Z) + \sigma[L_v(\boldsymbol{r})]\rho(Z) \text{ if } \sigma[L_v(\boldsymbol{r})] > 0 \\ 0 \text{ if } \sigma[L_v(\boldsymbol{r})] \le 0 \end{cases} \tag{6}$$

where

$$Z = \frac{\mu[L_v(\boldsymbol{r})] - L_v^+(\boldsymbol{r}) - \xi}{\sigma[L_v(\boldsymbol{r})]} \tag{7}$$

where $\mu[L_v(\boldsymbol{r})]$ is the mean prediction of the target function; $\sigma[L_v(\boldsymbol{r})]$ is the predicted standard deviation; $L_v^+(\boldsymbol{r})$ is the smallest (best) value of the target function observed so far; ξ is a parameter that controls the balance between exploration and exploitation. Here, the value of ξ is set to 0.01 [40]. $\phi(Z)$ is the standard normal cumulative distribution function, and $\rho(Z)$ is the standard normal probability density function.

It should be noted that Bayesian optimization infers the posterior distribution of the model's objective function (model output). The prior distribution is defined over the objective function itself and is updated after each iteration. Given that both the prior distribution and the likelihood of the objective function are assumed to be Gaussian, an analytical solution for the posterior distribution can be derived. This differs from studies like those by Chen et al. [37] and Xu et al. [38], which used Bayesian inference to update the posterior distribution of existing model parameters based on experimental data. In these cases, the prior distribution was defined over the model parameters, and directly calculating the posterior distribution of the model parameters was infeasible. Instead, these studies employed Markov Chain Monte Carlo (MCMC) simulations to estimate the posterior distribution.

The process for hyperparameter tuning using Bayesian optimization consists of the following steps:

(1) Initialization: Evaluate the target function using initial hyperparameters. These preliminary observations enable the GP model to form initial predictions and assess uncertainties related to the target function.
(2) Compute the acquisition function: Calculate the value of the acquisition function in the hyperparameter space. *EI* operates on the principle that the anticipated improvement is maximized at points where the predicted mean is lower than the current best observed value, and where there's a higher predicted uncertainty (higher standard deviation in new point predictions).
(3) Optimize the acquisition function: Conduct a global search in the hyperparameter space to identify points that maximize expected improvement, particularly focusing on hyperparameters that could substantially lower the validation loss.
(4) Sample new points: Once the group of hyperparameters that maximizes expected improvement is identified, evaluate the target function using these hyperparameters.
(5) Update the GP model: Add this new group of hyperparameters and its corresponding target function value to the observational dataset, then update the GP model to derive new predictions. The process then returns to step (2) and repeats until a termination criterion is met.

4. Implementation of model training and testing

This section details the systematic implementation of both MAML and ANN utilizing the Bayesian optimization technique. To ensure a fair comparison between MAML and ANN, the optimal set of hyperparameters for each model is identified individually. This approach is taken because the two models may exhibit distinct sensitivities to various hyperparameters. By optimizing hyperparameters for MAML and ANN independently, the full potential of each model under the most favorable conditions can be evaluated.

4.1. MAML using Bayesian optimization technique

The implementation of MAML in this study primarily utilizes the open-source machine learning library PyTorch (version 2.1.2+cpu) [41]. PyTorch, a library written in Python, provides extensive modules for deep and complex learning. Python code execution is conducted using Anaconda (version 23.7.4) [42]. Bayesian optimization is implemented using the *BayesianOptimization* library [40], which is an independent library from PyTorch. This library is recognized for its powerful capability in global optimization, especially effective for optimizing complex and expensive functions. A simplified workflow of the Bayesian optimization-based MAML on training tasks is depicted in Fig. 4. Upon identifying the optimal hyperparameter set, the corresponding model state is saved and used to evaluate the model's generalization ability on the testing task.

This workflow includes the following key aspects:

(1) Construction of multiple training tasks

For an effective MAML model, a substantial number of training tasks are essential, typically at least ten times the number of input features. Ideally, training tasks should be independent and identically distributed, drawn from an overall distribution. However, in scenarios with limited data, it's often necessary to create varied training tasks using the same sample pool. This approach enables MAML to learn broader features. In this study, considering variations in loading methods, boundary constraints, materials, and geometric dimensions across experiments, the training samples were divided into five subsets. For each training task, a training dataset with 5 randomly chosen samples from each subset was

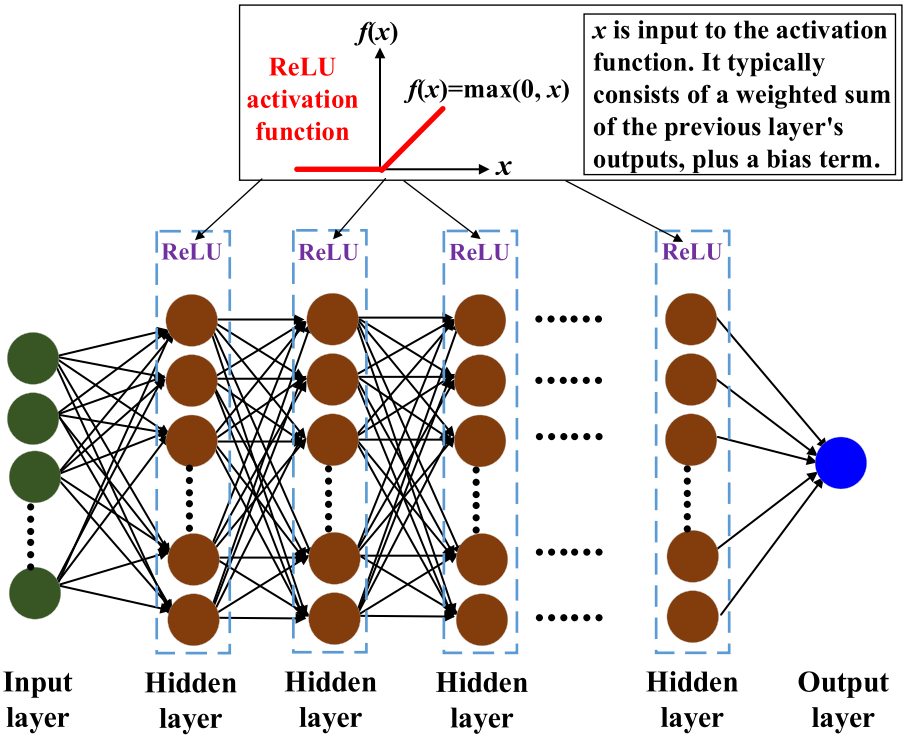


Fig. 5. The adopted neural network along with activation function for hidden layers.

generated, amounting to 25 samples in the training dataset per task. The validation dataset for each task included the unselected samples from each subset, making up 35 samples in total. After repeated sampling from these subsets, a total of 100 training tasks were generated. This methodology ensures that each task is derived from a consistent distribution, yet with potential overlaps between the training and validation datasets across different tasks, a permissible practice in MAML.

(2) Parameter initialization, loss and activation function determination

Model parameters were initialized using Xavier initialization (also known as Glorot initialization) [43], a common method for parameter initialization. This approach maintains a reasonable scale of activations and gradients during the forward and backward propagation in the network, thus avoiding issues like gradient vanishing or exploding. The loss function used for training, validation, and testing is the relative root mean square error (RRMSE), a commonly used loss function in regression tasks. It is expressed as:

$$RRMSE = \frac{\sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(M^{i}_{max,pre} - M^{i}_{max,obs}\right)^{2}}}{\mu\left(M_{max,obs}\right)} \tag{8}$$

where n is the number of samples; $M^{i}_{max,pre}$ and $M^{i}_{max,obs}$ are the predicted and observed maximum cyclic moments for the i-th sample, respectively; $\mu(M_{max,obs})$ is the mean of all observed sample values.

The adopted neural network is depicted in Fig. 5, in which the hidden layers use the Rectified Linear Unit (ReLU) [44] as the activation function. ReLU maintains a constant gradient of one in the positive region, preventing gradient vanishing issues, and outputs zero in the negative region, leading to sparse activation in the network and helping to mitigate overfitting. Compared to Sigmoid and Tanh functions, ReLU can accelerate the convergence speed of the neural network. For regression tasks, the neural network's output layer does not use an activation function, allowing the model's output to directly map to continuous numeric predictions.

(3) Model parameter updates

Model parameter updates were accomplished through the Adaptive Moment Estimation (Adam) [45] optimizer. The Adam optimizer merges the advantages of two extended gradient descent algorithms: Momentum and Root Mean Square Propagation (RMSprop), making it more effective and stable in handing non-stationary objectives. Adam's first momentum decay rate (β_1) computes the first-order moment (momentum) of gradients, while its second momentum decay rate ($\beta 2$) calculates the second-order moment (uncentered variance). The default settings in PyTorch for $\beta 1$ and $\beta 2$, which are 0.9 and 0.999 [41], respectively, were utilized in this study. Moreover, an L2 regularization (weight decay) parameter [46] was used in the Adam optimizer to mitigate the risk of overfitting during training. L2 regularization reduces model complexity by imposing a penalty term to the model's weights, thereby lowering overfitting.

(4) Hyperparameter boundaries, target function and Bayesian iteration

The hyperparameter for MAML ranges defined for the Bayesian optimization were set as follows: neurons per layer ranging from 10 to 70, number of layers between 1 and 20, inner learning rate from 0.00001 to 0.01, outer learning rate between 0.0001 and 0.1, meta batch size from 2 to 100, and a weight decay parameter within 0.00001 to 0.01. These ranges are deliberately chosen to be broad enough to encompass potential optimal values, while still maintaining a focus on computational efficiency. To simplify the Bayesian search process, the total number of meta epochs was fixed at 200, which was considered sufficient for the convergence of the MAML model's validation loss.

In the selection of the optimal set of hyperparameters, the primary metric is the minimal averaged validation loss on training tasks, while ensuring the training loss does not excessively surpass the validation loss, which could indicate underfitting. This approach was adopted because validation loss is a better indicator of the model's generalization capacity, whereas focusing solely on minimizing training loss could lead

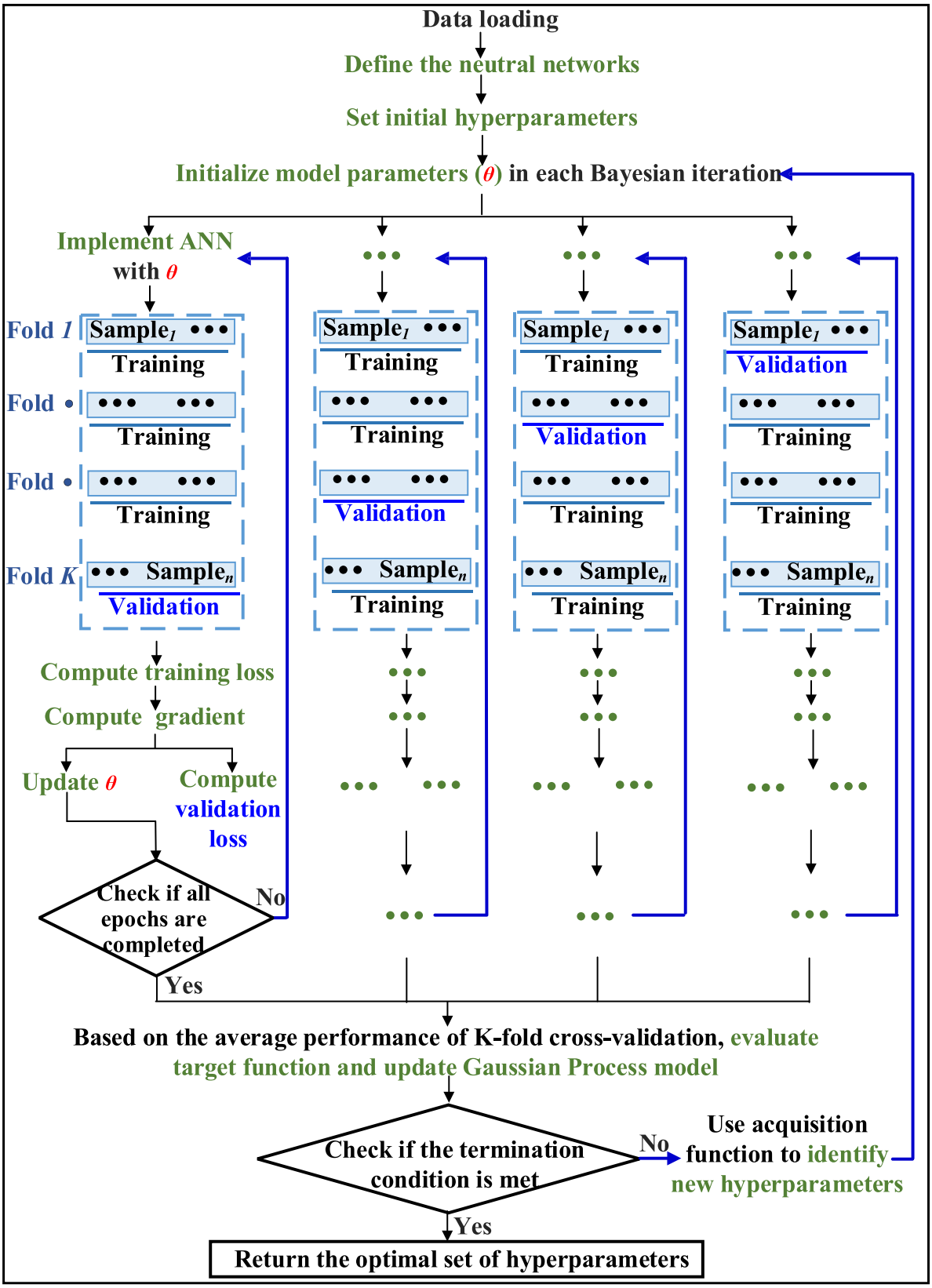


Fig. 6. Illustration of a simplified workflow for Bayesian optimization-based ANN on training samples.

to overfitting. Preventing overfitting is particularly vital as overfitted models generally exhibit poor performance on new data.

Within the Bayesian optimization process, the GP model updates after each invocation of the target function. Initially, 10 groups of hyperparameters were set, followed by 40 iterations. For each call, the minimum of the average validation loss during the meta-learning cycle serves as an observation point for the GP model to enhance its comprehension of the hyperparameter space. New hyperparameters are

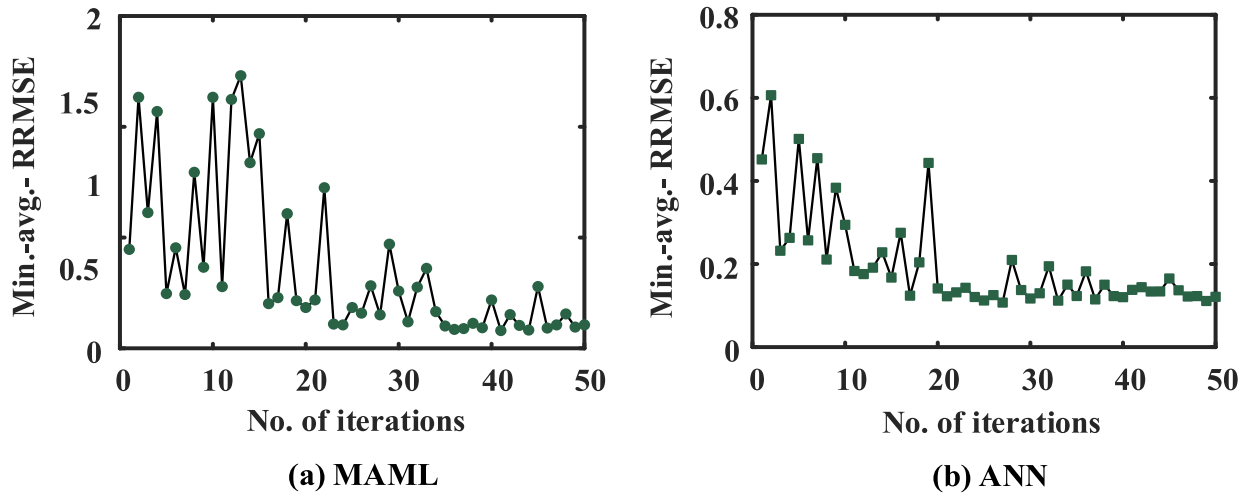


Fig. 7. The relationship between the number of Bayesian iterations and the minimum of the average validation loss for MAML and ANN.

selected based on the current state of the GP model, and the iterative process continues by repeatedly invoking the target function.

4.2. ANN using Bayesian optimization technique

A simplified workflow of the Bayesian optimization-based ANN on training tasks is depicted in Fig. 6. Besides the following aspects, the other implementation details of ANN are similar to MAML:

(1) ANN is within-task training (which can be considered as having only one training task), unlike the across-tasks training in MAML, and ANN does not have an inner loop.

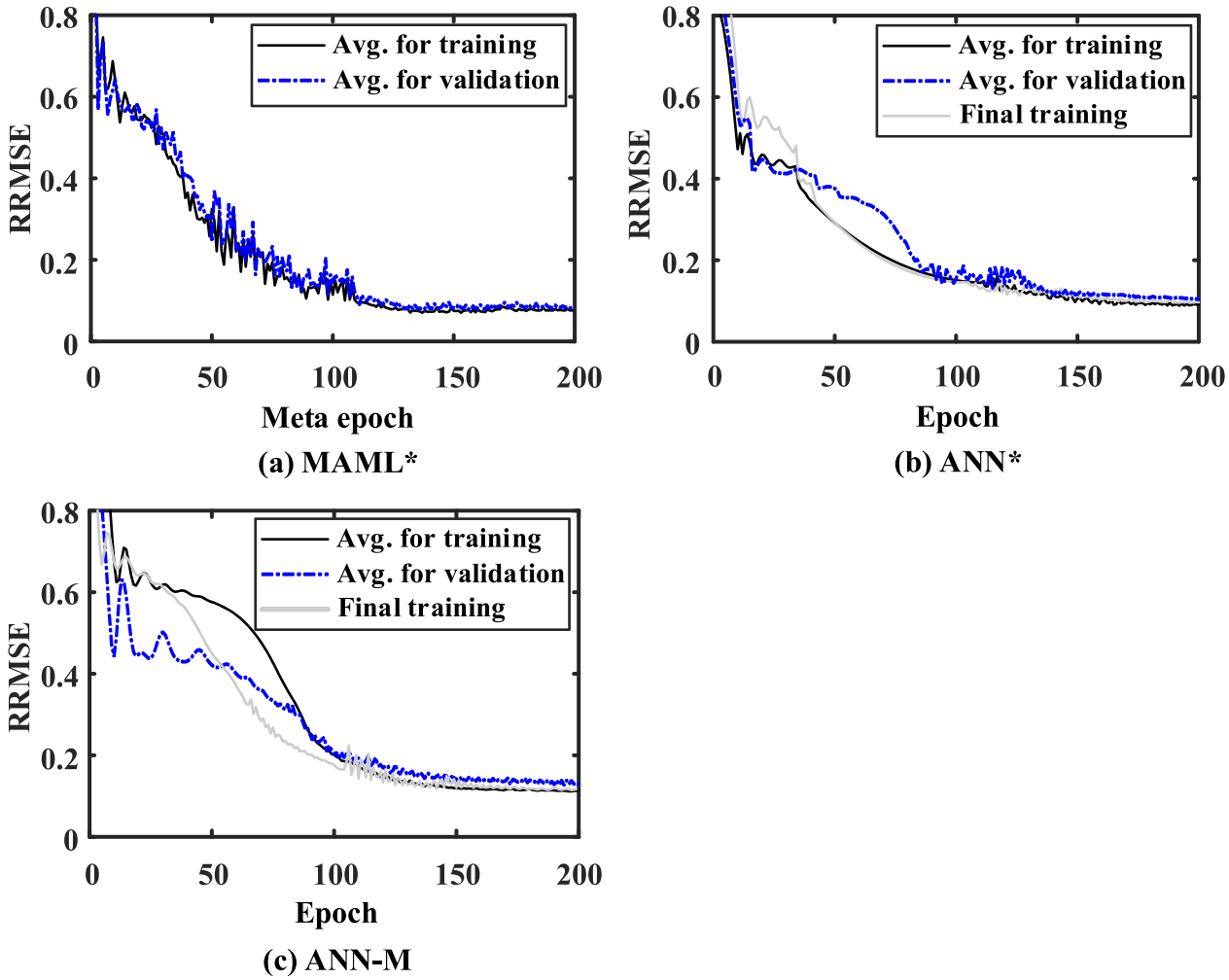


Fig. 8. Loss curves of MAML*, ANN* and ANN-M on the training samples.

Table 3
Statistical summary for the performance of MAML* on the training samples.

Metrics	Task_4		Task_5		Task_13		Task_18	
	Training	Validation	Training	Validation	Training	Validation	Training	Validation
μ	1.014	0.967	0.975	0.996	0.993	0.982	0.991	0.992
SD	0.083	0.089	0.082	0.078	0.094	0.096	0.071	0.103
CV	0.082	0.092	0.084	0.078	0.095	0.098	0.072	0.104
Max	1.166	1.152	1.092	1.145	1.182	1.234	1.150	1.173
Min	0.903	0.791	0.807	0.794	0.792	0.805	0.897	0.792
RRMSE	0.075	0.082	0.071	0.073	0.082	0.080	0.069	0.077
Metrics	Task_23		Task_43		Task_44		Task_57	
	Training	Validation	Training	Validation	Training	Validation	Training	Validation
μ	0.986	0.991	0.961	1.025	0.971	1.021	1.003	0.996
SD	0.093	0.081	0.080	0.091	0.085	0.093	0.097	0.101
CV	0.094	0.082	0.083	0.089	0.087	0.091	0.097	0.101
Max	1.166	1.214	1.143	1.199	1.166	1.177	1.265	1.278
Min	0.792	0.868	0.807	0.796	0.792	0.810	0.807	0.792
RRMSE	0.084	0.075	0.087	0.075	0.074	0.080	0.083	0.085
Metrics	Task_58		Task_70		Task_75		Task_89	
	Training	Validation	Training	Validation	Training	Validation	Training	Validation
μ	1.007	0.991	0.992	0.975	0.995	0.979	1.001	0.993
SD	0.098	0.107	0.092	0.086	0.084	0.093	0.095	0.106
CV	0.097	0.108	0.092	0.088	0.085	0.095	0.095	0.107
Max	1.214	1.276	1.166	1.142	1.158	1.165	1.231	1.279
Min	0.792	0.806	0.792	0.806	0.807	0.791	0.807	0.791
RRMSE	0.079	0.080	0.072	0.079	0.066	0.081	0.073	0.085

Metrics	Task_90		Task_94	
	Training	Validation	Training	Validation
μ	0.985	1.000	0.990	1.019
SD	0.079	0.098	0.090	0.102
CV	0.080	0.098	0.091	0.100
Max	1.158	1.184	1.265	1.288
Min	0.828	0.793	0.792	0.847
RRMSE	0.063	0.079	0.084	0.080

(2) ANN employs K-fold cross-validation, where the target function is the minimum of the average validation loss across all validations. It's noteworthy that MAML does not use K-fold cross-validation because each training task in MAML has its own training and validation samples, a structure that is incompatible with K-fold cross-validation.
(3) For ANN, the hyperparameters optimized via Bayesian optimization include: learning rate, L2 regularization parameter, K-fold, neurons per layer, and number of layers. The range for K was set from 2 to 15, and the learning rate ranged from 0.0001 to 0.1. Other hyperparameter ranges were kept the same as those for MAML.
(4) After performing K-fold cross-validation, ANN needs to be retrained on the entire training samples using optimal hyperparameters. This necessity arises because during the K-fold cross-validation process, the samples in one fold used for validation do not contribute to the model parameter updates. That means the K sets of model parameters are generated based on the samples from K-1 folds. It is worth noting that, due to its unique architecture, MAML does not require retraining because both training and validation samples for each task contribute to the model parameter updates.

5. Results of hyperparameter optimization and model performance evaluation

This section first presents the results of hyperparameter optimization using Bayesian optimization approach for MAML and ANN, followed by their performance on the training samples with the optimized hyperparameters. For a fair comparison, the performance of ANN that shares the same neural network architecture as MAML, including identical numbers of layers and nodes, is also evaluated. In addition to these three machine learning models, the prediction results of the design specification ASCE41 on the same training samples are also presented. Finally, to assess generalization capability, their performance on independent test samples is assessed.

5.1. Hyperparameter optimization results for MAML and ANN

The relationship between the number of iterations of Bayesian optimization (horizontal axis) and the minimum of the average validation loss (denoted by Min.-avg.-RRMSE for the vertical axis) for MAML and ANN is illustrated in Fig. 7 (a) and (b), respectively. It is observed that although the curve for MAML fluctuates more than that for ANN, both curves exhibit a similar overall trend. This trend is characterized by several pronounced peaks in Min.-avg.-RRMSE in the early stages (approximately 23 iterations for MAML and 19 for ANN), indicating the exploration of suboptimal hyperparameter groups. The peak value of MAML is more than twice that of ANN, which may be attributable to MAML being more sensitive to the hyperparameters. After early fluctuations, a noteworthy decrease in Min.-avg.-RRMSE is observed, with subsequent oscillations around lower loss values. This pattern suggests that the Bayesian algorithm might have identified a more suitable set of hyperparameters. As iterations continue, the curves show several marked fluctuations, possibly reflecting the algorithm's attempts at probing beyond the current optimal parameters in search of superior alternatives. The decline in Min.-avg.-RRMSE becomes more subtle as the optimization process continues. Despite some minor oscillations, a general trend of convergence is evident, indicating that the Bayesian optimization has likely identified an optimal or near-optimal set of hyperparameters within the defined space.

Upon completion of the iterations, Bayesian optimization identified

Table 4
Statistical summary for the performance of ANN* on the training samples.

Metrics	Folds0_123		Folds1_023		Folds2_013		Folds3_012		Final training
	Training	Validation	Training	Validation	Training	Validation	Training	Validation	
μ	0.997	1.029	1.022	1.061	1.014	0.956	1.005	0.998	0.990
SD	0.111	0.129	0.123	0.135	0.120	0.125	0.116	0.123	0.123
CV	0.111	0.125	0.120	0.127	0.119	0.131	0.116	0.124	0.124
Max	1.207	1.346	1.199	1.293	1.305	1.232	1.278	1.310	1.326
Min	0.805	0.774	0.785	0.822	0.770	0.760	0.810	0.810	0.786
RRMSE	0.091	0.105	0.093	0.103	0.090	0.111	0.091	0.100	0.097

Table 5
Statistical summary for the performance of ANN-M on the training samples.

Metrics	Folds0_123		Folds1_023		Folds2_013		Folds3_012		Final training
	Training	Validation	Training	Validation	Training	Validation	Training	Validation	
μ	0.997	1.060	1.016	1.113	1.019	1.023	0.986	0.973	1.027
SD	0.161	0.210	0.157	0.195	0.170	0.133	0.160	0.147	0.166
CV	0.161	0.198	0.154	0.175	0.167	0.130	0.162	0.151	0.162
Max	1.487	1.436	1.409	1.505	1.447	1.371	1.353	1.272	1.452
Min	0.792	0.786	0.746	0.840	0.780	0.838	0.725	0.772	0.772
RRMSE	0.118	0.132	0.109	0.140	0.115	0.102	0.118	0.128	0.117

Table 6
Statistical summary for the performance of ASCE41 on the training samples.

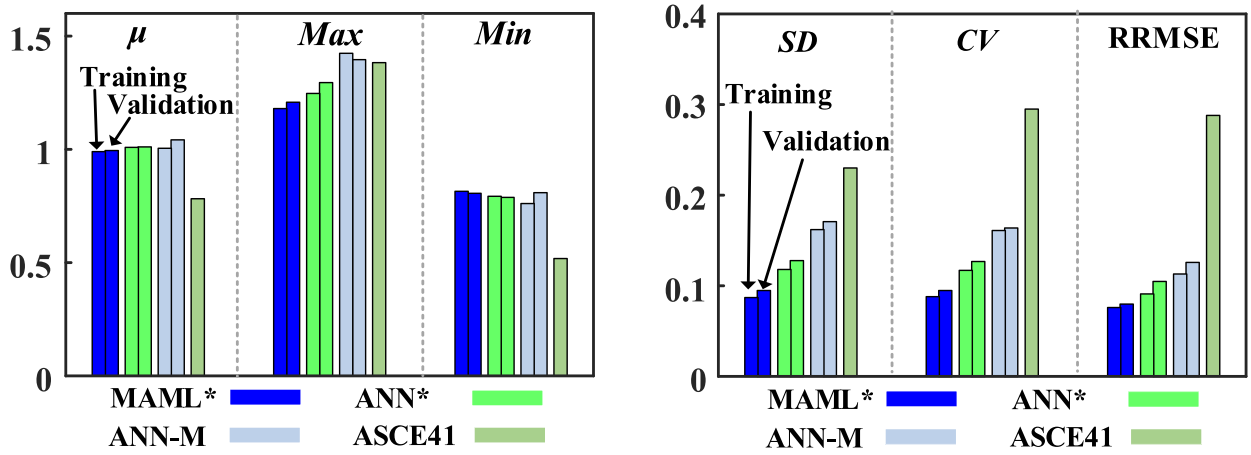
μ	*SD*	*CV*	*Max*	*Min*	RRMSE
0.782	0.230	0.295	1.383	0.518	0.288

the following as the optimal hyperparameter set for MAML: neurons per layer = 34, number of layers = 6, inner learning rate = 0.00038, outer learning rate = 0.00907, meta batch size = 14, weight decay parameter = 0.00017. The optimal hyperparameter set for ANN is: neurons per layer = 29, number of layers = 7, learning rate = 0.0046, K = 4, weight decay parameter = 0.00048.

5.2. Performance on training samples

The average training and validation loss curves of MAML using the optimized hyperparameter set (MAML*), ANN using the optimized hyperparameter set (ANN*), and ANN using the same neural network as optimized MAML (ANN-M) are presented in Fig. 8. MAML* demonstrates superior performance in both average training (0.076) and validation (0.080) losses. For ANN* and ANN-M, there are noticeable differences in average training and validation losses during the early stages of training, but these gaps narrow as training progresses. After stabilization, ANN* shows better performance with average training (0.091) and validation (0.105) losses lower than those of ANN-M (average training 0.113 and validation 0.126). For ANNs, the retrained training loss curves closely resemble the average training loss curves.

Statistical summaries for the performance of MAML*, ANN*, ANN-M, and ASCE41 are presented in Tables 3–6, respectively. For MAML*, results are shown from the final meta epoch across all 14 tasks (meta batch size = 14). For ANN* and ANN-M, results from the final epoch of 4-fold cross-validation (K = 4) and from the final epoch of retraining are presented. In Tables 4 and 5, the first number in the Fold names denotes the sequence number of the fold used for validation, while the last three numbers denote the sequence numbers of the folds used for training. The primary metric for evaluation is RRMSE, which offers a comprehensive performance evaluation, with lower values indicating smaller errors and better model performance. Five additional metrics related to the ratio of predicted to observed values, including mean (*μ*), standard deviation (*SD*), coefficient of variation (*CV*), maximum (*Max*), and minimum



(a) Metrics of *μ*, *Max* and *Min* (b) Metrics of *SD*, *CV* and RRMSE

Fig. 9. Bar plots for the average values of each metric for all models.

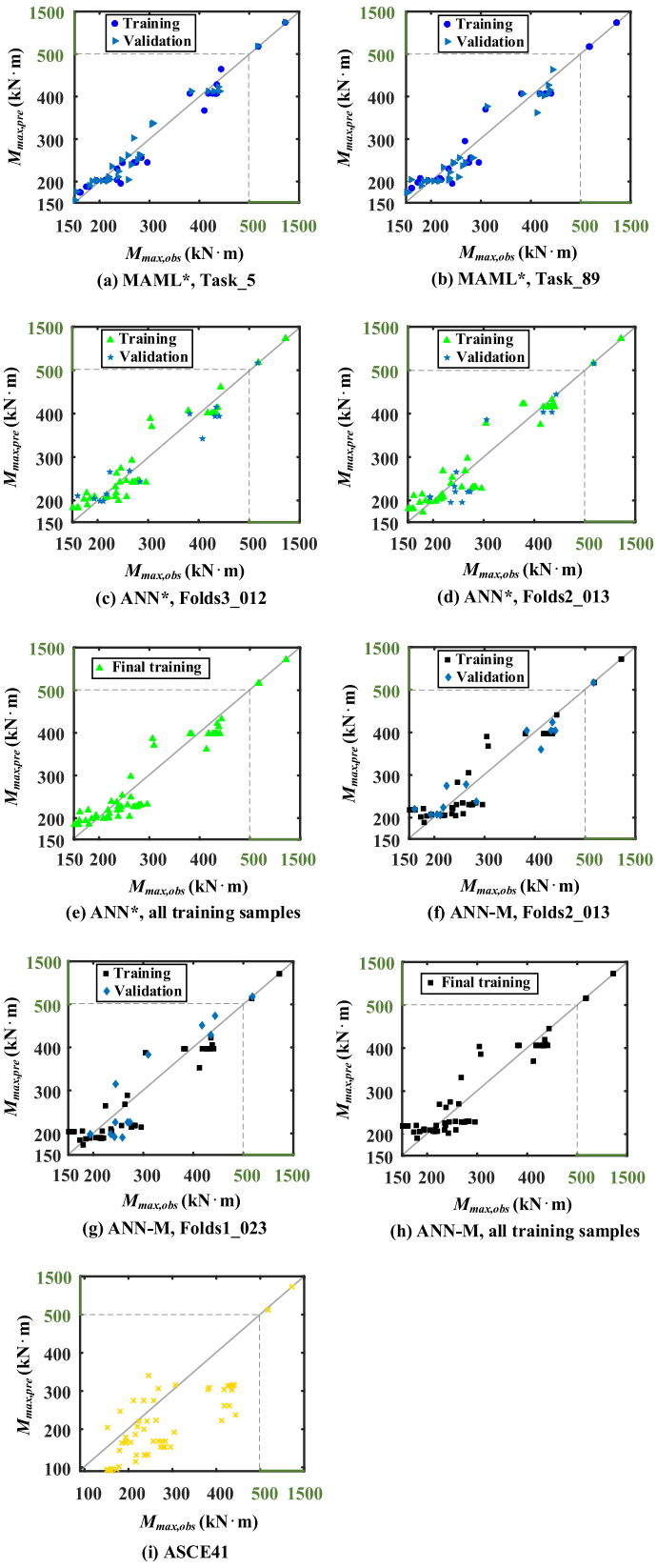


Fig. 10. Scatter plots for MAML*, ANN*, ANN-M and ASCE 41on the training samples.

(*Min*), are used to supplement the evaluation with RRMSE. μ assesses average predictive accuracy, while *SD* and *CV* evaluate stability, and *Max* and *Min* provide insight into the range of predictive performance. To facilitate comparison between different models, the average values of each metric for all models are additionally displayed in bar plots, as shown in Fig. 9 (a) and (b). For each of the three machine learning models, the metrics for training and validation are presented in two adjacent bars, in which the first and second bar represent the metric for

Table 7
Statistical Summary for the performance of MAML*, ANN*, ANN-M and ASCE41 on the testing samples.

Metrics	MAML*	ANN*	ANN-M	ASCE41
μ	0.981	1.026	1.083	0.863
SD	0.102	0.134	0.146	0.242
CV	0.104	0.131	0.134	0.280
$\varepsilon+$	0.864	0.793	0.807	0.588
$\varepsilon-$	1.165	1.208	1.270	1.177
RRMSE	0.095	0.141	0.167	0.200

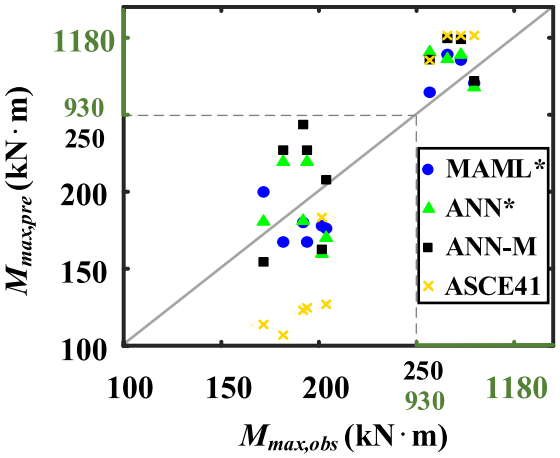


Fig. 11. Scatter plots for MAML*, ANN*, ANN-M and ASCE 41on the testing samples.

training and the validation, respectively.

It is observed that MAML* performs better than the other two machine learning models and the ASCE 41 model across all metrics. The superior performance of MAML* in metrics across various training tasks demonstrates its reliable predictive stability. Among the training tasks, MAML* stands out on Task_5 for its lowest validation RRMSE, indicating good predictions close to observed values, as evidenced in Fig. 10 (a). Despite exhibiting the highest validation RRMSE on Task_89, MAML* maintains commendable performance on this task, as depicted in Fig. 10 (b). For the two ANNs, ANN* slightly lags behind MAML* but it performs better than ANN-M. This is further illustrated by the scatter plots in Fig. 10 (c) and (d), which display the highest and lowest validation RRMSEs observed in ANN* during K-fold cross-validation. The discrepancy in validation RRMSE between the highest (0.140) and lowest (0.102) for ANN-M indicates high instability, with corresponding scatter plots in Fig. 10 (f) and (g). Notably, retraining of ANN* and ANN-M reveals a performance that aligns closely with their average outcomes in K-fold cross-validation, further evidenced by scatter plots in Fig. 10 (e) and (h), respectively. In contrast to the three machine learning models, ASCE41 exhibits a significantly higher RRMSE of 0.288 on training samples, indicating relatively larger prediction errors compared to the machine learning models, as evidenced in Fig. 10 (i). This, alongside other inflated metrics (e.g., *CV*=0.295), suggests that while traditional design standard-based methodologies have their place, machine learning models offer superior predictive capabilities.

5.3. *Performance on testing samples*

A statistical summary of the performance of MAML*, ANN*, ANN-M, and ASCE 41-17 on the testing task is shown in Table 7, with scatter plots corresponding to each method presented in Fig. 11. It is clear that MAML*'s predictions show remarkable proximity to observed results, significantly outperforming other methods. MAML*'s small loss (0.095) on the testing samples closely matches its performance on the training samples, demonstrating good generalization. This indicates that through learning across multiple tasks, MAML* can learn from limited samples and generalize to new samples effectively. Despite good training performance, ANN*'s testing loss (0.141) significantly increases, indicating its generalization ability is hindered by the limited training samples. It should be pointed out that, the performance of machine learning models on testing tasks is crucial as it reflects their ability to generalize to unseen data. Despite sharing the same neural network as MAML*, ANN-M's testing performance significantly lags behind MAML*, indirectly highlighting the superiority of MAML*'s learning strategy. Although ASCE41 shows improvement on the testing samples compared to its performance on training samples, its metrics still fall short of the three machine learning models, further demonstrating the advantages of machine learning models.

6. Interpretability of the model predictions

To enhance the interpretability of the machine learning model predictions, Shapley Additive Explanations (SHAP) [47], a game theory-based approach for explaining machine learning models, were employed to explain the contribution of each predictor variables

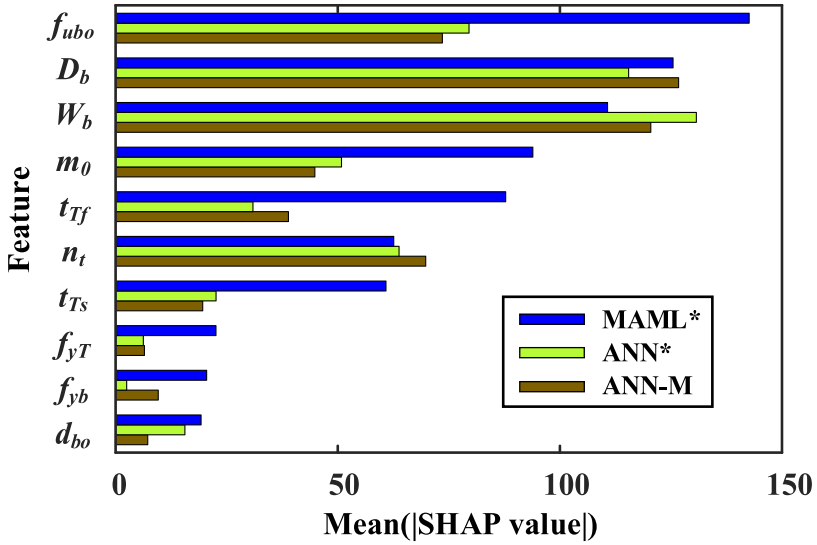


Fig. 12. The average absolute SHAP values by feature for MAML*, ANN* and ANN-M.

(features) to the model output. SHAP values are derived by considering all possible combinations of features and averaging the marginal contributions across these combinations. The specific calculation method can be found in [47]. SHAP values can be positive or negative, indicating whether the feature increases or decreases the prediction value respectively. The larger the absolute value of the SHAP, the more significant the feature's impact on the prediction.

Fig. 12 shows the average absolute SHAP values by feature for MAML*, ANN* and ANN-M. In the bar chart, the vertical axis lists the feature names, while the horizontal axis represents the average absolute SHAP value for each feature. Features are ranked based on their average contribution in MAML*, from highest to lowest. It can be seen that the average contribution ranking of features in ANN* and ANN-M displays notable variations compared to that in MAML*, with significant differences in the average absolute SHAP values for certain features (f_{ubo}, m_0, t_{Tf} and t_{Ts}). Specifically, MAML* exhibits higher average absolute SHAP values across a broader range of features (f_{ubo}, D_b, W_b, m_0, t_{Tf}, n_t and t_{Ts}). This is attributed to MAML*'s ability to identify features that are important across different tasks, resulting in higher average absolute SHAP values for these features. It reflects MAML*'s extensive adaptability to multiple tasks. A few features (W_b and D_b) in ANN* and ANN-M have considerably higher average absolute SHAP values compared to others. This occurs because ANN* and ANN-M focus on specific key features due to their training on a single task, which results in more concentrated SHAP values for a few features and lower absolute SHAP values for others. Furthermore, by analyzing the average absolute SHAP values across the three machine learning models, it can be inferred that the primary features influencing the cyclic resistance of steel bolted T-stub connections are f_{ubo}, D_b, W_b, m_0, t_{Tf} and n_t .

It should be noted that SHAP values are not comparable to the Pearson Correlation Coefficient (*PCC*, discussed in Section 2.2). *PCC* measures the linear relationship between features and the output variables in the samples used to train the model, which cannot capture nonlinear relationships and does not consider the correlation among multiple features. In contrast, SHAP values assess the relationship between features and the model's predictions, whether linear or nonlinear, and indirectly account for the interaction effects between features.

7. Conclusions

This research investigates the potential of employing the MAML in predicting the cyclic moment resistance of steel bolted T-stub connections. The main conclusions are as follows:

(1) Bayesian optimization proves to be a highly effective approach in identifying the optimal hyperparameter set for MAML. This optimal group of hyperparameters enables the MAML to achieve commendable accuracy in its predictions, evidenced by the low average training (0.076) and validation (0.080) losses.
(2) The optimized MAML (MAML*) outperforms both ANN with optimized hyperparameters (ANN*) and ANN with the same neural network structure as MAML* (ANN-M) across several metrics on the training samples. MAML*'s RRMSE ranges between 0.063 and 0.087 across various training tasks, demonstrating remarkable prediction stability and accuracy. While ANN* shows somewhat inferior training performance to MAML*, it still outperforms ANN-M. With an RRMSE of 0.288, the design specification ASCE41 exhibits relatively larger prediction errors on the training samples, underlining the significant improvements in predictive accuracy offered by machine learning models.
(3) MAML* exhibits good generalization ability on the testing samples, evidenced by a small RRMSE of 0.095. This is attributed to MAML*'s unique learning strategy, which enables effectively learning from limited data and robust generalization to new scenarios. In contrast, both ANN* and ANN-M face challenges in generalization ability. ANN* shows a higher RRMSE (0.141) on testing samples compared to training samples, indicating its generalization ability is hindered by the limited training samples. ANN-M's performance on the testing samples was even poorer, further highlighting the effectiveness of MAML*'s learning strategy. In contrast to the three machine learning models, ASCE 41 shows the weakest performance on the testing samples.
(4) The interpretability analysis using SHAP values reveals that the predictions of MAML* primarily depend on the features f_{ubo}, D_b, W_b, m_0, t_{Tf}, n_t and t_{Ts}, while the features W_b and D_b contribute significantly more to the predictions of ANN* and ANN-M compared to other features.

This study primarily focuses on MAML model's favorable generalization capabilities for small sample sizes. Future work will extend the application of the MAML model to other connections similar to steel bolted T-stub connections, aiming to further assess its rapid adaptability and generalization in new but related tasks.

CRediT authorship contribution statement

Yanfei Shen: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Mao Li:** Writing – review & editing. **Yong Li:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The authors acknowledge the financial support provided by the Natural Sciences and Engineering Research Council (NSERC) in Canada through the Discovery Grant (RGPIN-2017-05556).

Appendix

Table A1
Details of the collected samples for training and testing.

	Refs.	Specimen	Bolt				T-stub			Beam			Column			M_{max}
			Grade	N_t	d_{bo} (mm)	f_{ubo} (MPa)	$D_T \times W_T \times t_{Ts} \times t_{Tf}$ (mm)	f_{yT} (MPa)	f_{uT} (MPa)	$D_b \times W_b \times t_{bw} \times t_{bf}$ (mm)	f_{yb} (MPa)	f_{ub} (MPa)	$D_c \times W_c \times t_{cw} \times t_{cf}$ (mm)	f_{yc} (MPa)	f_{uc} (MPa)	(kN•m)
Training samples	[28]	FS-03	A490	8	22.2	1034	318 × 178 × 10 × 14	417	565	524 × 165 × 10 × 11	368	482	375 × 394 × 17 × 27	386	510	668.0
		FS-04	A490	8	25.4	1034	356 × 178 × 10 × 14	417	565	524 × 165 × 10 × 11	368	482	375 × 394 × 17 × 27	386	510	681.8
	[29]	TJL	10.9	4	20.0	1108	250 × 200 × 10 × 15	385	450	294 × 200 × 8 × 12	385	450	300 × 300 × 10 × 15	275	433	181.0
		TL	10.9	4	20.0	1108	250 × 200 × 12 × 19	385	450	294 × 200 × 8 × 12	385	450	300 × 300 × 10 × 15	275	433	152.3
		LL	10.9	4	20.0	1108	250 × 200 × 10 × 15	385	450	294 × 200 × 8 × 12	385	450	300 × 300 × 10 × 15	275	433	151.4
		TJH	10.9	4	20.0	1108	250 × 200 × 12 × 19	385	450	294 × 200 × 8 × 12	385	450	300 × 300 × 10 × 15	275	433	169.5
	[1]	SS-02a	A490	8	35.0	1246	510 × 340 × 22 × 50	293	445	600 × 250 × 12 × 16	293	445	800 × 450 × 35 × 63	293	445	1221.0
		SS-02b	A490	8	35.0	1246	510 × 340 × 22 × 50	293	445	600 × 250 × 12 × 16	293	445	800 × 450 × 35 × 63	293	445	1219.4
	[30]	Base	10.9	4	20.0	1108	250 × 200 × 10 × 15	263	441	294 × 200 × 8 × 12	263	441	300 × 300 × 10 × 15	263	441	193.3
		T1	10.9	4	20.0	1108	250 × 200 × 12 × 19	263	441	294 × 200 × 8 × 12	263	441	300 × 300 × 10 × 15	263	441	234.9
		T2	10.9	4	20.0	1108	250 × 280 × 12 × 19	263	441	294 × 200 × 8 × 12	263	441	300 × 300 × 10 × 15	263	441	257.4
		B9	10.9	4	24.0	1108	250 × 200 × 10 × 15	263	441	294 × 200 × 8 × 12	263	441	300 × 300 × 10 × 15	263	441	205.3
		TY1	10.9	4	20.0	1108	250 × 200 × 10 × 10	255	441	294 × 200 × 8 × 12	235	441	300 × 300 × 10 × 15	255	441	173.2
		TY2	10.9	4	20.0	1108	250 × 200 × 10 × 20	255	441	294 × 200 × 8 × 12	235	441	300 × 300 × 10 × 15	255	441	211.7
		TF1	10.9	4	20.0	1108	250 × 200 × 15 × 15	255	441	294 × 200 × 8 × 12	235	441	300 × 300 × 10 × 15	255	441	215.7
		TF2	10.9	4	20.0	1108	250 × 200 × 20 × 15	255	441	294 × 200 × 8 × 12	235	441	300 × 300 × 10 × 15	255	441	220.5
		BH1	10.9	4	20.0	1108	250 × 200 × 10 × 15	255	441	294 × 200 × 8 × 12	235	441	300 × 300 × 10 × 15	255	441	179.8
		BH2	10.9	4	20.0	1108	250 × 200 × 10 × 15	255	441	294 × 200 × 8 × 12	235	441	300 × 300 × 10 × 15	255	441	235.2
		CY1	10.9	4	20.0	1108	250 × 200 × 10 × 15	255	441	294 × 200 × 8 × 12	235	441	300 × 300 × 10 × 10	255	441	185.0
		BOD1	10.9	4	16.0	1108	250 × 200 × 10 × 15	235	441	294 × 200 × 8 × 12	235	441	300 × 300 × 10 × 15	235	441	193.8
		BOD2	10.9	4	24.0	1108	250 × 200 × 10 × 10	235	441	294 × 200 × 8 × 12	235	441	300 × 300 × 10 × 15	235	441	216.2
		MAT	10.9	4	20.0	1108	250 × 200 × 10 × 15	345	466	294 × 200 × 8 × 12	345	466	300 × 300 × 10 × 15	345	466	242.4
		BG	10.9	4	20.0	1108	250 × 200 × 10 × 15	235	441	294 × 200 × 8 × 12	235	441	300 × 300 × 10 × 15	235	441	194.0
	[31]	PJB12	10.9	4	22.0	1000	270 × 200 × 9 × 12	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	160.6
		PJB14	10.9	4	22.0	1000	270 × 200 × 9 × 14	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	236.7
		JBTT14-N	10.9	4	22.0	1000	270 × 200 × 9 × 14	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	217.9
		JBTD12-N	10.9	4	22.0	1000	270 × 200 × 9 × 12	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	159.9
		JCTT12	10.9	4	22.0	1000	270 × 200 × 9 × 12	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	150.8
		JBTT12-N	10.9	4	22.0	1000	270 × 200 × 9 × 12	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	153.8
		JBTD14-N	10.9	4	22.0	1000	270 × 200 × 9 × 14	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	178.3
		JCTD12	10.9	4	22.0	1000	270 × 200 × 9 × 12	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	161.7
		JATT18-N	10.9	4	22.0	1000	270 × 200 × 9 × 18	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	283.7
		JBTT16-N	10.9	4	22.0	1000	270 × 200 × 9 × 16	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	295.8
		JATD16-N	10.9	4	22.0	1000	270 × 200 × 9 × 16	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	244.1

(continued on next page)

Table A1 (*continued*)

	Refs.	Specimen	Bolt				T-stub			Beam			Column			M_{max}
			Grade	N_t	d_{bo} (*mm*)	f_{ubo} (*MPa*)	$D_T \times W_T \times t_{Ts} \times t_{Tf}$ (*mm*)	f_{yT} (*MPa*)	f_{uT} (*MPa*)	$D_b \times W_b \times t_{bw} \times t_{bf}$ (*mm*)	f_{yb} (*MPa*)	f_{ub} (*MPa*)	$D_c \times W_c \times t_{cw} \times t_{cf}$ (*mm*)	f_{yc} (*MPa*)	f_{uc} (*MPa*)	(*kN•m*)
		JBTD18-N	10.9	4	22.0	1000	270 × 200 × 9 × 18	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	276.2
		JATT16-N	10.9	4	22.0	1000	270 × 200 × 9 × 16	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	273.2
		JCTD16-N	10.9	4	22.0	1000	270 × 200 × 9 × 16	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	282.5
		JCTD18-N	10.9	4	22.0	1000	270 × 200 × 9 × 18	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	268.7
		JBTD16-N	10.9	4	22.0	1000	270 × 200 × 9 × 16	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	279.2
		JATD18-N	10.9	4	22.0	1000	270 × 200 × 9 × 18	257	409	350 × 175 × 7 × 11	276	436	300 × 300 × 10 × 15	293	450	256.5
	[33]	BASE	10.9	2	20.1	1040	250 × 200 × 10 × 16	235	375	400 × 200 × 8 × 12	235	375	500 × 300 × 10 × 16	235	375	426.5
		TPT1	10.9	2	20.1	1040	250 × 200 × 10 × 10	235	375	400 × 200 × 8 × 12	235	375	500 × 300 × 10 × 16	235	375	307.1
		TPT2	10.9	2	20.1	1040	250 × 200 × 10 × 20	235	375	400 × 200 × 8 × 12	235	375	500 × 300 × 10 × 16	235	375	435.1
		TWT1	10.9	2	20.1	1040	250 × 200 × 6 × 16	235	375	400 × 200 × 8 × 12	235	375	500 × 300 × 10 × 16	235	375	304.3
		TWT2	10.9	2	20.1	1040	250 × 200 × 14 × 16	235	375	400 × 200 × 8 × 12	235	375	500 × 300 × 10 × 16	235	375	437.9
		CPT1	10.9	2	20.1	1040	250 × 200 × 10 × 16	235	375	400 × 200 × 10 × 12	235	375	500 × 300 × 10 × 16	235	375	418.0
		CPT2	10.9	2	20.1	1040	250 × 200 × 10 × 16	235	375	400 × 200 × 10 × 20	235	375	500 × 300 × 10 × 16	235	375	432.2
		CWT1	10.9	2	20.1	1040	250 × 200 × 10 × 16	235	375	400 × 200 × 8 × 12	235	375	500 × 300 × 6 × 16	235	375	381.0
		CWT2	10.9	2	20.1	1040	250 × 200 × 10 × 16	235	375	400 × 200 × 8 × 12	235	375	500 × 300 × 14 × 16	235	375	435.1
		BPT1	10.9	2	20.1	1040	250 × 200 × 10 × 16	235	375	400 × 200 × 8 × 8	235	375	500 × 300 × 10 × 16	235	375	440.7
		BPT2	10.9	2	20.1	1040	250 × 200 × 10 × 16	235	375	400 × 200 × 8 × 16	235	375	500 × 300 × 10 × 16	235	375	383.9
		BOS1	10.9	2	20.1	1040	250 × 200 × 10 × 16	235	375	400 × 200 × 8 × 12	235	375	500 × 300 × 10 × 16	235	375	443.6
		BOS2	10.9	2	20.1	1040	250 × 200 × 10 × 16	235	375	400 × 200 × 8 × 12	235	375	500 × 300 × 10 × 16	235	375	412.3
		BWT2	10.9	2	20.1	1040	250 × 200 × 10 × 16	235	375	400 × 200 × 10 × 12	235	375	500 × 300 × 10 × 16	235	375	418.0
		BWT1	10.9	2	20.1	1040	250 × 200 × 10 × 16	235	375	400 × 200 × 6 × 12	235	375	500 × 300 × 10 × 16	235	375	429.4
	[34]	TF1	10.9	2	24.5	1040	200 × 400 × 13 × 12	335	380	450 × 200 × 8 × 12	235	310	350 × 350 × 12 × 16	235	310	237.7
		TF2	10.9	2	24.5	1040	200 × 400 × 13 × 26	335	380	450 × 200 × 8 × 12	235	310	350 × 350 × 12 × 16	235	310	246.0
		TW1	10.9	2	24.5	1040	200 × 400 × 10 × 21	335	380	450 × 200 × 8 × 12	235	310	350 × 350 × 12 × 16	235	310	224.4
		TW2	10.9	2	24.5	1040	200 × 400 × 16 × 21	335	380	450 × 200 × 8 × 12	235	310	350 × 350 × 12 × 16	235	310	263.2
		MAT	10.9	2	24.5	1040	200 × 400 × 13 × 21	345	391	450 × 200 × 8 × 12	345	391	350 × 350 × 12 × 16	345	391	268.0
Testing samples	[28]	FS-05	A490	8	22.2	1034	384 × 264 × 14 × 25	359	496	600 × 178 × 10 × 13	423	528	375 × 394 × 17 × 27	386	510	1043.5
		FS-06	A490	8	25.4	1034	356 × 264 × 14 × 25	359	496	600 × 178 × 10 × 13	423	528	375 × 394 × 17 × 27	386	510	1008.9
		FS-07	A490	8	22.2	1034	384 × 213 × 14 × 24	369	500	600 × 178 × 10 × 13	423	528	375 × 394 × 17 × 27	386	510	1077.5
		FS-08	A490	8	25.4	1034	432 × 213 × 14 × 24	371	500	600 × 178 × 10 × 13	376	473	375 × 394 × 17 × 27	386	510	963.6
	[29]	Base	10.9	4	20.0	1108	250 × 200 × 10 × 15	385	450	294 × 200 × 8 × 12	385	450	300 × 300 × 10 × 15	275	433	193.9
		TJ	10.9	4	20.0	1108	250 × 200 × 10 × 15	385	450	294 × 200 × 8 × 12	385	450	300 × 300 × 10 × 15	275	433	181.7
		TJS	10.9	8	20.0	1108	250 × 200 × 10 × 15	385	450	294 × 200 × 8 × 12	385	450	300 × 300 × 10 × 15	275	433	201.5
	[30]	C1	10.9	4	20.0	1108	250 × 200 × 10 × 15	263	441	294 × 200 × 8 × 12	263	441	350 × 350 × 12 × 19	263	441	203.7
		BH	10.9	4	20.0	1108	250 × 200 × 10 × 15	263	441	250 × 200 × 9 × 14	263	441	300 × 300 × 10 × 15	263	441	171.6
	[2]	TS-CYC 04	10.9	4	20.0	1000	318 × 257 × 10 × 25	295	520	269 × 132 × 7 × 10	396	540	203 × 200 × 9 × 15	406	522	191.9

Note: N_t: number of tension bolts on one T-stub; d_{bo}: diameter of the tension bolt shank; D: cross-section depth; W: cross-section width; t_w: thickness of the web (t_{Ts} for T-stem); t_f: thickness of the flange; f_y: yield strength; f_u: ultimate tensile strength.

References

[1] R. Herrera, C. Salas, J.F. Beltran, E. Nunez, Experimental performance of double built-up T moment connections under cyclic loading, J. Constr. Steel Res. 138 (2017) 742–749, https://doi.org/10.1016/j.jcsr.2017.08.022.

[2] V. F. Iannone, M. Latour, V. Piluso, G. Rizzano, Experimental analysis of bolted steel beam-to-column connections: component identification J. Earthq. Eng. 15 (2) (2011) 214–244, https://doi.org/10.1080/13632461003695353.

[3] ASCE 41-17, Seismic Evaluation and Retrofit of Existing Buildings, American Society of Civil Engineers, Reston, 2017.

[4] Y. Shen, M. Li, Y. Li, A reliable cyclic envelope model for partially-restrained steel beam-column bolted T-stub joints based on experimental data, Thin Walled Struct. 200 (2024) 111951, https://doi.org/10.1016/j.tws.2024.111951.

[5] W. Chen, R. Fediuk, J. Yu, et al., Performance prediction and analysis of engineered cementitious composites based on machine learning, Dev. Built Environ. 18 (2024) 100459, https://doi.org/10.1016/j.dibe.2024.100459.

[6] J. Liu, S. Li, J. Guo, et al., Machine learning (ML) based models for predicting the ultimate bending moment resistance of high strength steel welded I-section beam under bending, Thin Walled Struct. 191 (2023) 111051, https://doi.org/10.1016/j.tws.2023.111051.

[7] S. Hu, S. Zhu, W. Wang, Machine learning-driven probabilistic residual displacement-based design method for improving post-earthquake repairability of steel moment-resisting frames using self-centering braces, J. Build. Eng. 61 (2022) 105225, https://doi.org/10.1016/j.jobe.2022.105225.

[8] K. Jiang, O. Zhao, Unified machine-learning-assisted design of stainless steel bolted connections, J. Constr. Steel Res. 211 (2023) 108155, https://doi.org/10.1016/j.jcsr.2023.108155.

[9] J.M. Reinosa, A. Loureiro, R. Gutierrez, M. Lopez, Artificial neural network prediction of the initial stiffness of semi-rigid beam-to-column connections, Structures 56 (2023) 104904, https://doi.org/10.1016/j.istruc.2023.104904.

[10] S.N.R. Shah, N.R. Sulong, A. El-Shafie, New approach for developing soft computational prediction models for moment and rotation of boltless steel connections, Thin Walled Struct. 133 (2018) 206–215, https://doi.org/10.1016/j.tws.2018.09.032.

[11] A. Vettoruzzo, MR. Bouguelia, J. Vanschoren, et al., Advances and challenges in meta-learning: a technical review, IEEE Trans. Pattern. Anal. Mach. Intell. (2024) 1–20, https://doi.org/10.1109/TPAMI.2024.3357847.

[12] J. Gu, Y. Wang, Y. Chen, et al., Meta-learning for low-resource neural machine translation, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, 2018, https://doi.org/10.48550/arXiv.1808.08437.

[13] A. Entezami, H. Sarmadi, B. Behkamal, Long-term health monitoring of concrete and steel bridges under large and missing data by unsupervised meta learning, Eng. Struct. 279 (2023) 115616, https://doi.org/10.1016/j.engstruct.2023.115616.

[14] Z. Almheiri, M. Meguid, T. Zayed, Failure modeling of water distribution pipelines using meta-learning algorithms, Water Res. 205 (2021) 117680, https://doi.org/10.1016/j.watres.2021.117680.

[15] C. Finn, P. Abbeel, S. Levine, Model-agnostic meta-learning for fast adaptation of deep networks, in: Proceedings of the 34th International Conference on Machine Learning, Sydney, Australia, PMLR 70, 2017, https://doi.org/10.48550/arXiv.1703.03400.

[16] A. Antoniou, H. Edwards, A. Storkey, How to train your MAML, in: Proceedings of the 7th International Conference on Learning Representations, New Orleans, LA, USA, ICLR, 2019, https://doi.org/10.48550/arXiv.1810.09502.

[17] A. Fallah, A. Mokhtari, A. Ozdaglar, Personalized federated learning with theoretical guarantees: a model-agnostic meta-learning approach, Adv. Neural Inf. Process. Syst. 33 (2020), https://doi.org/10.48550/arXiv.2002.07948.

[18] V.L. Tran, D.K. Thai, D.D. Nguyen, Practical artificial neural network tool for predicting the axial compression capacity of circular concrete-filled steel tube columns with ultra-high-strength concrete, Thin Walled Struct. 151 (2020) 106720, https://doi.org/10.1016/j.tws.2020.106720.

[19] C. Huang, Y. Li, Q. Gu, et al., Machine learning–based hysteretic lateral force-displacement models of reinforced concrete columns, J. Struct. Eng. 148 (3) (2022) 04021291, https://doi.org/10.1061/(ASCE)ST.1943-541X.0003257.

[20] V.L. Tran, S.E. Kim, Efficiency of three advanced data-driven models for predicting axial compression capacity of CFDST columns, Thin Walled Struct. 152 (2020) 106744, https://doi.org/10.1016/j.tws.2020.106744.

[21] R. Shamass, F.P.V. Ferreira, V. Limbachiya, et al., Web-post buckling prediction resistance of steel beams with elliptically-based web openings using Artificial Neural Networks (ANN), Thin Walled Struct. 180 (2022) 109959, https://doi.org/10.1016/j.tws.2022.109959.

[22] A.E. Kurtoglu, E. Casanova, C. Graciano, Artificial intelligence-based modeling of extruded aluminum beams subjected to patch loading, Thin Walled Struct. 179 (2022) 109673, https://doi.org/10.1016/j.tws.2022.109673.

[23] S.M. Mojtabaei, J. Becque, I. Hajirasouliha, et al., Predicting the buckling behaviour of thin-walled structural elements using machine learning methods, Thin Walled Struct. 184 (2023) 110518, https://doi.org/10.1016/j.tws.2022.110518.

[24] M. Mundt, S. Majumder, S. Murali, et al., Meta-learning convolutional neural architectures for multi-target concrete defect classification with the concrete defect bridge image dataset, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019. *http://arxiv.org/abs/1904.08486*.

[25] X. Wang, Y. Jin, S. Schmitt, et al., Recent advances in Bayesian optimization, ACM Comput. Surv. 55 (13s) (2023) 1–36, https://doi.org/10.1145/3582078.

[26] A.H. Victoria, G. Maragatham, Automatic tuning of hyperparameters using Bayesian optimization, Evol. Syst. 12 (2021) 217–223, https://doi.org/10.1007/s12530-020-09345-2.

[27] P.I. Frazier. A Tutorial on Bayesian Optimization. arXiv:1807.02811, 2018. 10.48550/arXiv.1807.02811.

[28] J.M. Smallidge. Behavior of bolted beam-to-column T-stub connections under cyclic loading. Master's thesis, Georgia Institute of Technology, Atlanta (GA), 1999.

[29] X. Li. Research on the mechanical properties of steel beam-to-column T-stub semi-rigid connection with test and simulation. Master's thesis, Guangxi University, Nanning, China, 2016. (in Chinese).

[30] X. Zheng. Study on the mechanical behaviors of steel beam-to-column T-stub connection under cyclic load. Master's thesis, Guangxi University, China, 2013. (in Chinese).

[31] X. Bu. Research on seismic behaviors of spatial connections with T-stub in steel frames. Doctoral thesis, Wuhan University of Technology, Wuhan, China, 2018. (in Chinese).

[32] Steel Construction Manual, American Institute of Steel Construction, Chicago, IL, 2005.

[33] X. Cao, J. Hao, C. Shen, Analysis of the anti-seismic capacity of T-stub beam-column joint of the steel frame under cyclic load, Steel Constr. 23 (109) (2008) 30–38. Chinese.

[34] Z. Fang, Master's thesis, China University of Petroleum (East China), Qingdao, China, 2011.

[35] J. Benesty, J. Chen, Y. Huang, I. Cohen, Pearson correlation coefficient. noise reduction in speech processing, in: Springer Topics in Signal Processing, 2, Springer, Berlin, Heidelberg, 2009, https://doi.org/10.1007/978-3-642-00296-0_5.

[36] C.K. Williams, C.E. Rasmussen, Gaussian Processes for Machine Learning, MIT press, Cambridge, MA, 2006, https://doi.org/10.7551/mitpress/3206.001.0001.

[37] W. Chen, J. Xu, M. Dong, et al., Data-driven analysis on ultimate axial strain of FRP-confined concrete cylinders based on explicit and implicit algorithms, Compos. Struct. 268 (2021) 113904, https://doi.org/10.1016/j.compstruct.2021.113904.

[38] J. Xu, W. Chen, C. Demartino, et al., A Bayesian model updating approach applied to mechanical properties of recycled aggregate concrete under uniaxial or triaxial compression, Constr. Build. Mater. 301 (2021) 124274, https://doi.org/10.1016/j.conbuildmat.2021.124274.

[39] D.K. Duvenaud. Automatic model construction with Gaussian processes. Doctoral thesis, University of Cambridge, Cambridgeshire, England, 2014.

[40] R. Martinez-Cantin, BayesOpt: a Bayesian optimization library for nonlinear optimization, experimental design and bandits, J. Mach. Learn. Res. 15 (1) (2014) 3735–3739.

[41] A. Paszke, S. Gross, F. Massa, et al., PyTorch: an imperative style, high-performance deep learning library, Adv. Neural Inf. Process. Syst. 32 (2019), https://doi.org/10.48550/arXiv.1912.01703.

[42] D. Rolon-Mérette, M. Ross, T. Rolon-Mérette, et al., Introduction to anaconda and python: installation and setup, Quant. Methods Psychol 16 (5) (2016) S3–S11, https://doi.org/10.20982/tqmp.16.5.S003.

[43] G. Xavier, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in: Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, Sardinia, Italy, JMLR Workshop and Conference Proceedings, 2010, pp. 249–256.

[44] V. Nair, G.E. Hinton, Rectified linear units improve restricted boltzmann machines, in: Proceedings of the 27th International Conference on Machine Learning (ICML-10), Haifa, Israel, 2010.

[45] D.P. Kingma, J. Ba. Adam: a method for stochastic optimization. arXiv:1412.6980, 2014. 10.48550/arXiv.1412.6980.

[46] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: Proceedings of the 7th International Conference on Learning Representations, New Orleans, LA, USA, ICLR 2019, 2019, https://doi.org/10.48550/arXiv.1711.05101.

[47] S.M. Lundberg, S.I. Lee, A unified approach to interpreting model predictions, Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, 2017. https://doi.org/10.48550/arXiv.1705.07874.